EQUIPMENTSDEKHO ECOSYSTEM

Partners Brand CRM System Architecture & Technical Manual

Comprehensive human-written technical reference for the enterprise B2B partner operations platform, multi-tenant RBAC, geographic territory routing, dynamic permissions engine, and API integration architecture.

APPLICATION NAME
partners-brand (Vite + React 19)

SYSTEM ARCHITECTURE
Single Page Application (SPA) REST Client

CORE TARGET USERS
Brand Principals (Admin) & Regional Executives (Sub-Admins)

BACKEND API BASE
https://partners.equipmentsdekho.com/apis/api

RELEASE VERSION
1.0.0 (Production Verified)

DOCUMENT DATE
October 2026

Table of Contents

1. Executive Summary & Business Objectives — Page 3
2. High-Level System Architecture — Page 3
3. Technology Stack & Complete Dependency Matrix — Page 4
4. Complete Project Directory Hierarchy & Code Anatomy — Page 5
5. Authentication, Session & Security Architecture — Page 7
6. Multi-Tenant Role-Based Access Control (RBAC) System — Page 8
7. Dynamic Navigation & Route Guard Engine — Page 9
8. CRM Operations Engine & Business Workflows — Page 10
9. Geographic Territory Management & Workload Auto-Assignment Engine — Page 12
10. Sub-Admin Operations & Dynamic Permission Configuration Matrix — Page 13
11. API Integration Layer & Complete Network Specification — Page 14
12. State Management Architecture & React Contexts — Page 16
13. UI/UX Design System & Common Component Library — Page 17
14. End-to-End Sequence & Data Flow Diagrams — Page 18
15. Environment Setup, Local Development & Build Verification — Page 19
16. Production Deployment & Optimization Recommendations — Page 20

1. Executive Summary & Business Objectives

The **EquipmentsDekho Partners Brand Portal** is a specialized enterprise SaaS platform built for industrial machinery manufacturers, brand principals, and authorized equipment distribution networks across India.

In the heavy machinery marketplace, buyers require high-touch sales interactions, technical specifications, and official quotation dispatches. This platform enables brand partners to ingest high-intent buyer inquiries in real time, route them instantly to field sales operators based on geographic territory boundaries, and manage the complete sales lifecycle from first customer inquiry to deal closure.

Key Business Drivers:

- **Instant Inbound Demand Ingestion:** Captures equipment inquiries and quotation requests directly from the EquipmentsDekho marketplace without manual re-entry.
- **Territory Integrity:** Strict boundary enforcement ensures regional dealers and field operators only access leads originating in their assigned states and cities.

- **Server-Driven Dynamic Capabilities:** Menus, permissions, and module views are not hardcoded on the client; they adapt dynamically based on server-side permission contracts.
- **Workload-Balanced Auto-Routing:** Prevents sales burnout and lead stagnation by routing unassigned leads to qualified Sub-Admins with the lowest open deal burden.
- **Tamper-Resistant Audit Trail:** Every call, note, meeting, quotation dispatch, and assignment override is preserved in an immutable chronological timeline.

2. High-Level System Architecture

The system utilizes a modern **Decoupled Single Page Application (SPA) Architecture** built on React 19, powered by Vite 8, and styled with Tailwind CSS v4. The client communicates with the EquipmentsDekho RESTful backend services using Bearer token-authenticated HTTP requests.

ARCHITECTURAL BLOCK REPRESENTATION

Presentation Layer	Control & State Layer	Data Access Layer	External Infrastructure
React 19 Components - Responsive Layouts - DataTable & Filters - Lead Drawer & Timeline - Modals & Skeletons - Tailwind CSS v4 Themes	**Context State Engine** - AuthContext (Session) - PermissionContext (RBAC) - CRMContext (Pipeline) - ThemeContext (Dark/Light) - React Router v7 Guards	**Services & Repositories** - authApi, inquiryApi - productQuotation - subAdminService - crmRepository (Cache) - Axios HTTP Client	**EquipmentsDekho Backend** - RESTful API Services - JWT Authentication - Lead Ingestion Engine - Multipart Uploads - Production Database

3. Technology Stack & Complete Dependency Matrix

Every package in the application has been chosen for high performance, tree-shaking capability, and minimal runtime overhead. Below is the production dependency specification:

Dependency Name	Version	Category	Technical Rationale & Responsibility
`react`	19.2.6	Core Framework	Latest React runtime utilizing Concurrent Mode, optimized reconciler, and modern state hooks.
`react-dom`	19.2.6	Rendering Target	DOM renderer for React 19 web applications.
`vite`	8.0.12	Build & Bundler	ESM development server with instantaneous HMR and Rollup production compilation.
`tailwindcss`	4.3.1	Styling Engine	Next-generation utility-first CSS engine integrated via `@tailwindcss/vite`.
`react-router-dom`	7.18.0	Routing Engine	Declarative client-side routing, nested layout outlets, and search param synchronization.
`axios`	1.18.0	Network Client	Promise-based HTTP client equipped with request authorization and response error interceptors.
`react-hook-form`	7.79.0	Form Handling	Uncontrolled form state management ensuring sub-millisecond input performance.
`react-hot-toast`	2.6.0	Notifications	Lightweight toast notifications featuring programmatic dismissal and deduplicated IDs.
`lucide-react`	1.21.0	Iconography	Complete SVG icon library maintaining consistent design across light and dark themes.
`react-select`	5.10.2	Select UI	Accessible searchable select and multi-select components for territory configuration.
`dotenv`	17.4.2	Configuration	Environment variable resolution supporting cross-environment deployment parameters.
`eslint`	10.3.0	Code Quality	Static code analysis enforcing React hooks rules and clean ES module conventions.

4. Complete Project Directory Hierarchy & Code Anatomy

The codebase follows a modular, feature-oriented structure with clean boundaries between UI components, business logic services, data access repositories, and configuration rules:

```
partners-brand/
├── .env                        # Base API environment configuration
├── .gitignore                  # Git exclusion rules
├── eslint.config.js            # ESLint flat configuration
├── index.html                  # Application entry HTML container
├── package.json                # Project dependencies, scripts, and engine metadata
├── tailwind.config.js          # Tailwind styling themes, fonts, and animation extensions
├── vite.config.js              # Vite bundler, React plugin & Tailwind Vite plugin
└── src/
```

```
    ├── main.jsx                      # React 19 root bootstrap
    ├── App.jsx                       # Master Application component, provider composition & Toast config
    ├── index.css                     # Global Tailwind stylesheets and font definitions
    │
    ├── api/                          # HTTP Communication Engine
    │   ├── axios.js                  # Axios instance with request/response interceptors & auto-refresh/401
redirect
    │   └── apiRequest.js             # Resilient wrapper normalizing promises, payload parsing & error formats
    │
    ├── endPoints/                    # Network Configuration
    │   └── APIEndpoints.js           # Declarative mapping of backend API endpoints and HTTP methods
    │
    ├── services/                     # Business Logic & Backend Service Integrations
    │   ├── authApi.js                # Login, registration, profile fetch, and session logout
    │   ├── dashboardApi.js           # Real-time dashboard KPI metrics and activity counters
    │   ├── inquiryApi.js             # Customer inquiries pagination and retrieval
    │   ├── productQuotation.js       # Feature equipment quote requests
    │   ├── profile.js                # Brand profile update service (multipart/form-data)
    │   ├── premiumBrands.js          # Brand catalogue services
    │   ├── subAdminService.js        # Sub-Admin retrieval, role mapping, and normalization
    │   ├── subAdminPermissionService.js # Role permission assignment (POST /add-premium-brand-role-permission)
    │   ├── permissionManagementService.js # Permission definitions registry (create/edit/list)
    │   └── subAdminSidebarService.js # Sub-Admin permitted sidebar dynamic loader (POST /sub-admin-sidebar)
    │
    ├── repositories/                 # Data Access Layer & Abstraction
    │   ├── subAdminRepository.js     # API transport layer for Sub-Admin queries
    │   ├── subAdminPermissionRepository.js # API transport for permissions assignment
    │   ├── permissionManagementRepository.js # API transport for permission definitions
    │   └── local/                    # Local storage repository & memory mock cache
    │       └── crmRepository.js      # CRUD repository for Leads, Quotations, Customers, Activities & Team
    │
    ├── context/                      # React Context State Providers
    │   ├── AuthContext.jsx           # Global authentication state, JWT storage, role normalization & session
check
    │   ├── PermissionContext.jsx     # Dynamic permissions loader for Admin and Sub-Admin roles
    │   ├── CRMContext.jsx            # Lead lifecycle state, territory visibility filter, auto-assigner,
activities
    │   └── ThemeContext.jsx          # Dark / Light theme persistence and HTML class synchronizer
    │
    ├── hooks/                        # Custom Reusable React Hooks
    │   ├── useBrand.js               # Convenience hook extracting brand ID, brand name, and metadata
    │   ├── useDataTable.js           # Sorting, pagination, and multi-field search for tabular views
    │   ├── usePermissions.js         # Permission evaluations, module access checks, and territory validation
    │   ├── useSubAdmins.js           # Data-fetching hook managing Sub-Admin lists, loading, and error states
    │   └── useSubAdminPermissions.js # Sub-Admin permission assignment state, dirty checking & submit handler
    │
    ├── routes/                       # Routing Configuration
    │   └── AppRoutes.jsx             # Complete route hierarchy, public/private gates, backward-compatibility
redirects
    │
    ├── layouts/                      # Layout Templates
    │   └── DashboardLayout.jsx       # Persistent responsive frame: Sidebar, TopHeader, and content Outlet
    │
    ├── components/                   # UI Component Ecosystem
    │   ├── common/                   # 30+ Atomic & Molecular Design System Components
    │   ├── crm/                      # LeadDetail, ActivityTimeline, AssignmentSelector, TerritorySelector
    │   ├── guards/                   # PrivateRoute, PublicRoute, PermissionRoute, PermissionGate
    │   ├── layout/                   # Sidebar, TopHeader, ThemeToggle, UserMenu
    │   └── permissions/              # PermissionFormModal (dynamic definition add/edit)
    │
    ├── pages/                        # View Pages & Route Containers
    │   ├── Dashboard.jsx             # Executive KPI analytics dashboard
    │   ├── CustomerEnquiry.jsx       # Inbound customer equipment inquiries table with drawer & CSV export
    │   ├── FeatureEquipmentQuotes.jsx# Product-specific quotation request registry with pipeline enrichments
    │   ├── Profile.jsx               # Brand Partner profile view and edit modal (multipart update)
    │   ├── crm/                      # AllLeads.jsx, FollowUps.jsx
```

```
    │   ├── permissions/              # PermissionManagement.jsx
    │   ├── reports/                  # CRMReports.jsx
    │   └── subadmins/                # SubAdmins.jsx, SubAdminDetail.jsx, SubAdminPermissions.jsx
    │
    └── utils/                        # Pure Helper Functions & Algorithms
        ├── accessRules.js            # Territory match evaluation, record view/update check & eligibility
engine
        ├── permissionResolver.js     # Slug-to-module and slug-to-actions normalizer
        ├── roleUtils.js              # Strict user_type normalizer (ADMIN | SUB_ADMIN | null)
        └── errorHandler.js           # API error payload parser extracting validation messages
```

5. Authentication, Session & Security Architecture

Authentication is managed through a centralized state provider in `AuthContext.jsx`, interacting directly with `authApi.js` and the Axios network interceptor layer.

5.1 Authentication Flow:

Step 1: Credential Submission — POST

User submits username and password via `Login.jsx` to endpoint `/premium-brand-login`.

↓

Step 2: Token Issuance & Role Normalization — SECURITY

Backend responds with JWT token and user record. `roleUtils.normalizeUserType()` validates `user_type`. Unknown roles are immediately rejected.

↓

Step 3: Storage & Interceptor Injection — STORAGE

Token is stored in `localStorage`. The Axios request interceptor attaches `Authorization: Bearer <token>` to all subsequent requests.

↓

Step 4: Startup Session Validation & 401 Guard — VALIDATION

On page reload, `fetchProfile()` calls `POST /profile`. Any 401 response wipes credentials and forces redirection to `/login` without infinite reload loops.

> **CRITICAL SECURITY IMPLEMENTATION RULE**
>
> The application **strictly refuses** to default missing or malformed user roles to Admin. If a user record lacks an explicit `user_type: "admin"` or `"sub_admin"`, the normalized role evaluates to `null`, and protected route access is immediately blocked.

6. Multi-Tenant Role-Based Access Control (RBAC) System

The system enforces strict operational separation between two fundamental roles:

Dimension	Brand Admin (Principal)	Sub-Admin (Regional Operator)
Geographic Scope	All Indian States & Cities (Nationwide visibility).	Strictly restricted to assigned States & Cities.
Navigation Menus	Loads dynamic CRM modules plus full Operations suite (Sub-Admins, Permissions, Reports).	Dynamically resolved strictly from `POST /sub-admin-sidebar` response.
Team Management	Can create, edit, activate/deactivate Sub-Admins, and configure their permission matrix.	Zero access to team management pages.

Dimension	Brand Admin (Principal)	Sub-Admin (Regional Operator)
Lead Assignment	Can assign to any operator, trigger auto-assignment, or confirm out-of-territory overrides.	Can only manage leads assigned to them or within their territory boundaries.
Reporting & Analytics	Full access to state-wise demand distribution and operator conversion metrics.	No access to executive reporting suite.

7. Dynamic Navigation & Route Guard Engine

Instead of hardcoding client-side routes, navigation items are built dynamically from server responses:

7.1 Slug-to-Module Resolution Engine (`permissionResolver.js`)

The backend sends permission tags as human titles or slugs (e.g. `"Customer Enquiry"`, `"quotes_enquiry"`, `"Leads"`). The resolver normalizes these strings to prevent breakage from backend naming variations:

```
export const resolveSlugToModule = (slug = "") => {
  if (!slug) return null;
  const clean = String(slug).trim().toLowerCase().replace(/[^a-z0-9]/g, "");

  if (clean.includes("enquir") || clean.includes("inquir")) return MODULES.CRM_ENQUIRIES;
  if (clean.includes("quote")  || clean.includes("quotation")) return MODULES.CRM_QUOTES;
  if (clean.includes("lead")) return MODULES.CRM_LEADS;
  if (clean.includes("followup") || clean.includes("follow")) return MODULES.CRM_FOLLOWUPS;
  if (clean.includes("report")) return MODULES.REPORTS;
  if (clean.includes("profile")) return MODULES.PROFILE;
  if (clean.includes("dashboard")) return MODULES.DASHBOARD;

  return slug;
};
```

7.2 Route Protection Guards

- `PrivateRoute` : Ensures a valid token and recognized role exist in session. If absent, redirects to `/login` .
- `PublicRoute` : Prevents authenticated users from seeing guest pages (redirects to `/dashboard`).
- `PermissionRoute` : Evaluates if the authenticated user possesses the specific module permission. Supports an `adminOnly` prop that immediately blocks Sub-Admins.
- `PermissionGate` : In-component conditional wrapper that renders buttons or actions only if the user has granular action permissions (e.g. `quotes:export`).

8. CRM Operations Engine & Business Workflows

8.1 Leads Management & Unified Pipeline (`AllLeads.jsx`)

The master leads register aggregates all prospective equipment buyers into a high-performance interactive table:

- **Tabs:** *All Leads* (complete visible dataset), *Unassigned Leads* (inbound queue requiring action), and *My Leads* (leads assigned to the logged-in operator).
- **Multi-Field Search:** Real-time filtering across customer name, phone, lead code, machine interest, city, and state.
- **Status Filters:** Filters across standardized statuses: `New` , `Assigned` , `Contacted` , `Follow-up` , `Quote Sent` , `Won` , and `Lost` .
- **Exporting:** Instant client-side CSV generation matching active query filters.

8.2 Customer Enquiries Workflow (`CustomerEnquiry.jsx`)

Direct integration with `POST /premium-brand-quotes-enquiry`. Features pagination controls, quick phone-calling triggers (`tel:`), and one-click lead drawer inspection.

8.3 Feature Equipment Quotation Requests (`FeatureEquipmentQuotes.jsx`)

Direct integration with `POST /get-feature-equipment-quotes`. Enriches product-specific quotation requests with CRM deal stages and pricing discussions.

8.4 360° Lead Detail Drawer (`LeadDetail.jsx`)

Sliding off-canvas drawer that presents an operational cockpit for any lead:

- **Activity Timeline:** Chronological history displaying calls, emails, notes, meetings, and stage changes.
- **Customer History:** Lifetime value and past inquiries associated with the customer's phone number.
- **Log Activity Tab:** Instant form to log call notes, meeting summaries, or internal remarks.
- **Schedule Follow-Up Tab:** Date and time scheduler that automatically populates the `FollowUps.jsx` pipeline.

9. Geographic Territory Management & Workload Auto-Assignment Engine

Lead routing is governed by an automated, workload-aware decision engine implemented in `utils/accessRules.js`:

AUTO-ASSIGNMENT LOGIC FLOW

1. Extract Lead Location: Identify the target `state` and `city` from inbound inquiry data.

2. Query Candidate Pool: Scan all Sub-Admins in the current brand account.

3. Evaluate Operational Eligibility: Candidate must meet three conditions:
- Account status is `active`
- Has view AND update permissions for this CRM type (e.g. `leads:view` + `leads:update`)
- Territory coverage: Operator's assigned territory state matches lead state, AND (cities include `"ALL"` OR lead city).

4. Workload Balancing Calculation: For all eligible candidates, compute active open burden:

```
openCount = leads.filter(l => l.assigned_to === member.id && l.status !== "won" && l.status !== "lost").length
```

5. Final Execution: Automatically assign the lead to the operator with the minimum active count. Record the automated routing decision in the lead activity timeline.

Admin Manual Territory Override

If a Brand Admin manually assigns a lead to an operator whose territories do not cover the lead's state/city, the system displays an **Override Confirmation Dialog**. Upon confirmation, the assignment proceeds and is logged with a permanent audit record:

> *"Admin territory override assignment by [Admin Name]. Reason: Out-of-territory manual assignment override confirmed."*

10. Sub-Admin Operations & Dynamic Permission Configuration Matrix

Brand Admins have granular control over operator accounts and access levels (`src/pages/subadmins/`):

- **Roster View:** Instant search across operator username, email, phone, company, and GST.
- **Account Lifecycle:** Activation toggle to suspend or restore operator access instantly.
- **Dynamic Permission Matrix (`SubAdminPermissions.jsx`):** Pulls live permission types from `POST /get-premium-brand-permissions` and allows toggling capabilities per operator.
- **Submission Payload:** When saved, it formats selected permissions into a canonical name array:

```
// POST /add-premium-brand-role-permission
{
  "role_id": "14",
  "permission_name": [
```

```
    "Customer Enquiry",
    "Quotes",
    "Leads",
    "Follow-ups"
  ]
}
```

11. API Integration Layer & Complete Network Specification

All network communication is centralized through `src/endPoints/APIEndpoints.js` and executed via the custom Axios instance in `src/api/axios.js`.

Endpoint Constant	Relative URL	Method	Content-Type	Functional Purpose
`login`	`/premium-brand-login`	POST	application/json	Authenticates brand principal or sub-admin. Returns JWT & profile.
`register`	`/premium-brand-register`	POST	MULTIPART	Registers brand partner with company, phone, GST, and address.
`profile`	`/profile`	POST	application/json	Retrieves authenticated user profile and account details.
`logout`	`/logout`	POST	application/json	Revokes active session on the backend.
`premiumBrandQuotesEnquiry`	`/premium-brand-quotes-enquiry`	POST	MULTIPART	Fetches paginated customer equipment inquiries for current brand.
`getFeatureEquipmentQuotes`	`/get-feature-equipment-quotes`	POST	MULTIPART	Retrieves equipment quotation requests and pricing inquiries.
`update_Profile`	`/update-profile`	POST	MULTIPART	Updates brand partner company profile, phone, GST, and address.
`getDashboardData`	`/get-dashboard-data`	POST	MULTIPART	Fetches dashboard counts, active metrics, and recent inquiries.
`getSubAdmins`	`/get-subadmins`	POST	application/json	Returns all Sub-Admins registered under the authenticated brand.
`getPremiumBrandPermissions`	`/get-premium-brand-permissions`	POST	application/json	Retrieves dynamic permission definitions list from server.
`addPremiumBrandPermission`	`/premium-brand-add-permission`	POST	application/json	Creates a new system permission definition.
`editPremiumBrandPermission`	`/premium-brand-edit-permission`	POST	application/json	Updates an existing permission definition name and slug.
`addPremiumBrandRolePermission`	`/add-premium-brand-role-permission`	POST	application/json	Assigns permitted permission names list to a Sub-Admin role ID.
`subAdminSidebar`	`/sub-admin-sidebar`	POST	application/json	Returns permitted sidebar menu strings for Sub-Admin role ID.

Global Error Handling & Toast Deduplication (`axios.js`)

The response interceptor formats errors and deduplicates toasts using message hashes:

- `HTTP 401 Unauthorized:` Clears localStorage session, triggers toast "*Session expired*", and forces redirect to `/login`.
- `HTTP 403 Forbidden:` Displays "*You are not authorized*" alert.
- `HTTP 422 Validation Error:` Parses nested `errors` payload and displays the first field validation error message.
- `HTTP 500+ Server Error:` Displays graceful server error notice without crashing client components.

12. State Management Architecture & React Contexts

State is structured into four focused contexts composed cleanly in `src/App.jsx`:

Context Provider	Key State Variables	Exposed Methods & Actions
`ThemeContext`	`theme` (`"dark"` | `"light"`)	`toggleTheme()`, `setTheme()`. Persists to `localStorage` and toggles HTML root class.
`AuthContext`	`token`, `user`, `role`, `isAdmin`, `isSubAdmin`, `isAuthenticated`, `loading`	`login(token, user, role)`, `logout()`, `fetchProfile()`. Normalizes roles safely.
`PermissionContext`	`adminPermissionDefinitions`, `subAdminSidebarItems`, loading and error states	`refreshAdminPermissions()`, `refreshSubAdminSidebar()`. Powers dynamic navigation.
`CRMContext`	`leads`, `unassignedLeads`, `quotations`, `teamMembers`, `customers`, `loading`	`assignLead()`, `autoAssignLead()`, `updateLeadStatus()`, `scheduleFollowup()`, `logActivity()`, `createQuotation()`.

13. UI/UX Design System & Common Component Library

Over 30 atomic and molecular components reside in `src/components/common/` ensuring accessibility, consistency, and zero layout shift:

Component Name	Component Type	Key Features & Accessibility Attributes
`DataTable`	Organism	Column-based sorting, custom cell renderers, empty state fallback, and pagination footer.
`Modal`	Molecule	Accessible backdrop, ESC key dismissal, focus trap, and enter/leave transitions.
`Drawer`	Organism	Off-canvas slide-over drawer for deep record inspection without leaving tabular context.
`ConfirmDialog`	Molecule	Destructive action confirmation dialogue with distinct danger and primary action styling.
`TableToolbar`	Molecule	Unified search input, filter trigger, CSV export button, and total count indicator.
`Skeletons`	Atom	CardSkeleton and TableSkeleton components preventing layout shifts while fetching data.
`AppErrorBoundary`	Guard	Top-level React error boundary displaying an elegant recovery card on uncaught render errors.

14. End-to-End Sequence & Data Flow Diagrams

Sub-Admin Dynamic Navigation Sequence

1. Sub-Admin logs in → Receives `user_type: "sub_admin"` and `id: 14`.
2. `PermissionContext` resolves role and dispatches `POST /sub-admin-sidebar` with `{ role_id: 14 }`.
3. Backend responds with permitted menu titles: `["Customer Enquiry", "Quotes"]`.
4. `mapDefinitionToNavItem()` normalizes titles to internal modules and routes.
5. Sidebar renders **Dashboard**, permitted CRM items, and **Profile**. Unpermitted items remain hidden.

15. Environment Setup, Local Development & Build Verification

15.1 Environment Configuration

Create a `.env` file in the repository root:

```
# EquipmentsDekho Partners Backend API URL (without trailing slash)
VITE_API_URL=https://partners.equipmentsdekho.com/apis/api
```

15.2 CLI Commands Reference

```
# 1. Install all dependencies
npm install

# 2. Launch development server with instantaneous HMR
npm run dev

# 3. Execute strict ESLint code quality checks
npm run lint

# 4. Compile optimized production build into dist/ directory
npm run build

# 5. Preview production build locally
npm run preview

# 6. Run production preview accessible over network on port 4000
npm run start
```

15.3 Build Verification Status

> **Production Build Verification Passed:**
> Executed `npm run build` successfully using Vite 8 and Rollup. Output generated in `dist/`:
> • `dist/index.html` (0.82 kB)
> • `dist/assets/index-[hash].css` (76.22 kB | gzip: 12.25 kB)
> • `dist/assets/index-[hash].js` (645.13 kB | gzip: 189.49 kB)
> • Zero syntax errors, zero missing imports, zero circular dependency failures.

16. Production Deployment & Optimization Recommendations

16.1 Nginx Static Hosting Configuration

```
server {
    listen 80;
    server_name partners.equipmentsdekho.com;
    root /var/www/partners-brand/dist;
    index index.html;

    # Gzip compression
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml;

    location / {
```

```
        try_files $uri $uri/ /index.html;
    }

    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

16.2 Strategic Engineering Recommendations

1. **Route-Level Code Splitting (`React.lazy`):** Split heavy pages (`CustomerEnquiry.jsx` , `FeatureEquipmentQuotes.jsx` , `CRMReports.jsx`) using dynamic imports to reduce initial bundle size below 200 kB.
2. **IndexedDB Offline Caching:** Integrate Dexie.js for client-side CRM caching so field operators can view lead contact details and past notes during temporary network outages.
3. **Real-Time SSE / WebSocket Alerts:** Implement Server-Sent Events to automatically trigger new lead notifications without requiring manual table refreshes.
4. **E2E Playwright Suite:** Automate critical test flows (Admin permission assignment, out-of-territory override confirmation, and quotation PDF generation).
